SIGNALDESK

Incident management, from local app to GitOps

Original React + FastAPI + PostgreSQL application with a tested delivery path through CI, containers, Terraform, Kubernetes, observability, and ArgoCD.

PREPARED CAPSTONE

LIVE DEPLOYMENT PENDING

An incident workflow that closes the loop

Each record carries service, severity, status, owner, description, and timestamps.

OPEN

Declare and assign

INVESTIGATING

Investigate impact

MITIGATED

Restore service

RESOLVED

Record closure

Dashboard

Status totals, recent incidents, search, filters, and a responsive detail drawer.

Recovery loop

Resolved incidents can reopen; invalid transitions return HTTP 409.

Three layers, one incident record

Local Compose is verified; the same application images are prepared for Kubernetes.

Browser and UI

React + Vite dashboard
Responsive CSS
Nginx runtime
/api routed to FastAPI

→

API and workflow

FastAPI REST endpoints
SQLAlchemy ORM
Readiness and /metrics
Alembic migrations

→

Persistent data

PostgreSQL incidents table
Create/update timestamps
Resolved timestamp
PVC in Kubernetes

Local: docker compose up --build | Cluster: NLB -> ingress-nginx -> frontend / backend -> PostgreSQL

The development gates have real local results

These checks verify code and containers; they do not prove a GitHub or EKS run.

API tests

8 isolated pytest cases across health, incident CRUD, workflow rules, and metrics.

Browser tests

4 Chrome checks at desktop and mobile widths, including API failure states.

Compose

Frontend, backend, and PostgreSQL ran together; app images ran as non-root.

The local /metrics endpoint returned a live request counter. No Prometheus target or Grafana panel has been verified in EKS.

Quality gates before image publication

Push to main is configured to trigger the workflow. A real GitHub run is pending.

01 VERIFY

pytest + Vite build

02 BUILD

Backend + frontend images

03 SCAN

Trivy HIGH / CRITICAL

04 PUBLISH

GHCR full SHA tags

Failing tests or a HIGH/CRITICAL image finding stops publication. Actions URL, pipeline scan output, and GHCR package screenshots are PENDING.

Terraform prepares the AWS foundation

A dummy-credential, refresh-free plan validated the graph; no AWS resources were created.

Network

VPC
2 public subnets
2 private subnets
Internet gateway + NAT

Compute

EKS cluster
Managed worker node group
2 desired t3.large nodes
Pod Identity Agent

Storage

EBS CSI add-on
gp3 StorageClass
PostgreSQL PVC
Prometheus and Grafana PVCs

LOCAL PLAN: 28 to add, 0 to change, 0 to destroy | LIVE PLAN, APPLY, CONSOLE, DESTROY: PENDING

Replicas, routing, and one migration owner

The Helm chart is rendered and validated locally; live Pods and Services remain pending.

Frontend

2 replicas
Nginx container
ClusterIP Service
Serves the React bundle

Backend

2 replicas minimum
CPU HPA up to 5
ClusterIP Service
/ready checks PostgreSQL

PostgreSQL

Single StatefulSet
Persistent gp3 claim
Connection in Secret
Back up before teardown

A pre-install / pre-upgrade Job runs Alembic once before the backend rolls. Replicas do not race to migrate the schema.

One internet-facing NLB, two app paths

Controller chart 1.14.0 uses app v2.14.0 and the matching v2.14.0 IAM policy.

AWS Load Balancer

NLB Service class
Internet-facing scheme
IP targets in the VPC

→

ingress-nginx

Receives public HTTP
Routes on path
Service gets NLB hostname

→

Application

/ -> React frontend
/api -> FastAPI backend
Both Services are ClusterIP

Ingress browser proof is PENDING. This demo path is HTTP; configure domain and TLS before sensitive public use.

From /metrics to a live Grafana panel

The metric contract and dashboard are prepared. Cluster scrape and visual proof are pending.

FastAPI

Request, latency, creation, transition counters

ServiceMonitor

Scrape backend /metrics every 30s

Prometheus

Discover the app across namespaces

Grafana

Request rate and p95 latency panels

Required live proof: Prometheus target UP and a populated SignalDesk application metric panel after real API traffic.

Helm demo, then ArgoCD ownership

The handoff prevents two controllers from continuously overwriting the same app resources.

1 Manual Helm

Install the app with a published SHA. Capture helm list and healthy Pods.

2 Handoff

Uninstall only the app release. Keep DB, Secrets, and monitoring.

3 ArgoCD

Sync chart from Git. PreSync Job migrates; tracked SHA values roll Pods.

The ArgoCD Application is prepared, but Synced/Healthy status and a GitOps update are PENDING LIVE proof.

Prepared configuration is not live proof

The grading checklist tracks every screenshot, URL, and terminal capture still needed.

Locally verified

8 pytest cases and 4 Chrome checks
Three-service Compose stack
Non-root application images
Terraform fmt/validate/offline plan
Helm render and schema checks

Pending external evidence

Actions + GHCR SHA packages
Real AWS plan/apply/destroy
Healthy EKS, Pods, Ingress
Prometheus UP + Grafana data
ArgoCD Synced + GitOps change

No screenshots, cloud results, or successful pipeline are fabricated in this deck.

The mandatory video starts with these slides

Then show the complete live path, with actual evidence on screen.

Video order

1. Present slide deck first
2. Incident workflow and tests
3. Push, Actions, Trivy, GHCR
4. Terraform and AWS Console
5. Helm, Ingress, monitoring, ArgoCD

Evidence to preserve

Commit history and test output
Running Pods, Services, Helm list
Public browser route
Prometheus target + Grafana panel
ArgoCD update and teardown

The video is PENDING until a real deployment exists. The rubric awards zero for the presentation module without it.

A traceable path needs clear gates and owners

Next: repository access, a verified pipeline, explicit AWS approval, then live evidence.

Quality before release

Tests and frontend build gate image creation. Trivy gates both image pushes.

One owner per resource

Terraform owns AWS; Helm bootstraps controllers; ArgoCD owns the app after handoff.

Operations matter

Guard cluster context, keep Secrets external, migrate once, monitor live, and destroy paid resources.

The remaining work is verification in real GitHub and AWS environments, not another offline rendering pass.